

Product Requirement Document (PRD)

InterioOS — AI Interior Design Project Manager

Version: 1.0 **Date:** September 13, 2026 **Prepared by:** Areeba Saleem (Member 6 — UI/UX Lead)

1. Overview

InterioOS is a multi-agent AI system that automates the early-stage planning of an interior design project. A user enters a plain-language requirement (e.g. "*modern bedroom, 8 lakh budget*"), and a pipeline of specialized AI agents generates a design concept, cost estimate, bill of quantities (BOQ), project timeline, and a consolidated final report — all in one run.

The system is built as a Final Year Project (FYP), divided across a 6-member team, using beginner-friendly tools: Python, basic LangGraph, the Claude API for each agent, and Streamlit for the user interface.

2. Problem Statement

Early-stage interior design planning normally requires separate manual effort for concept design, cost estimation, material listing, and scheduling — usually done by different people or tools, with no single view tying them together. InterioOS solves this by chaining specialized AI agents into one pipeline so a single input produces a complete, structured project plan.

3. Goals & Objectives

- Let a user submit one plain-text requirement and get a complete design plan without manual coordination between separate tools.
- Demonstrate a working multi-agent architecture using LangGraph, with each agent responsible for one stage of the plan.
- Keep the entire stack simple enough for a beginner-level team to build and understand (no database, no frontend framework, no Docker).
- Present results in a clear, organized web interface — not just raw text output.

Out of Scope (for this version)

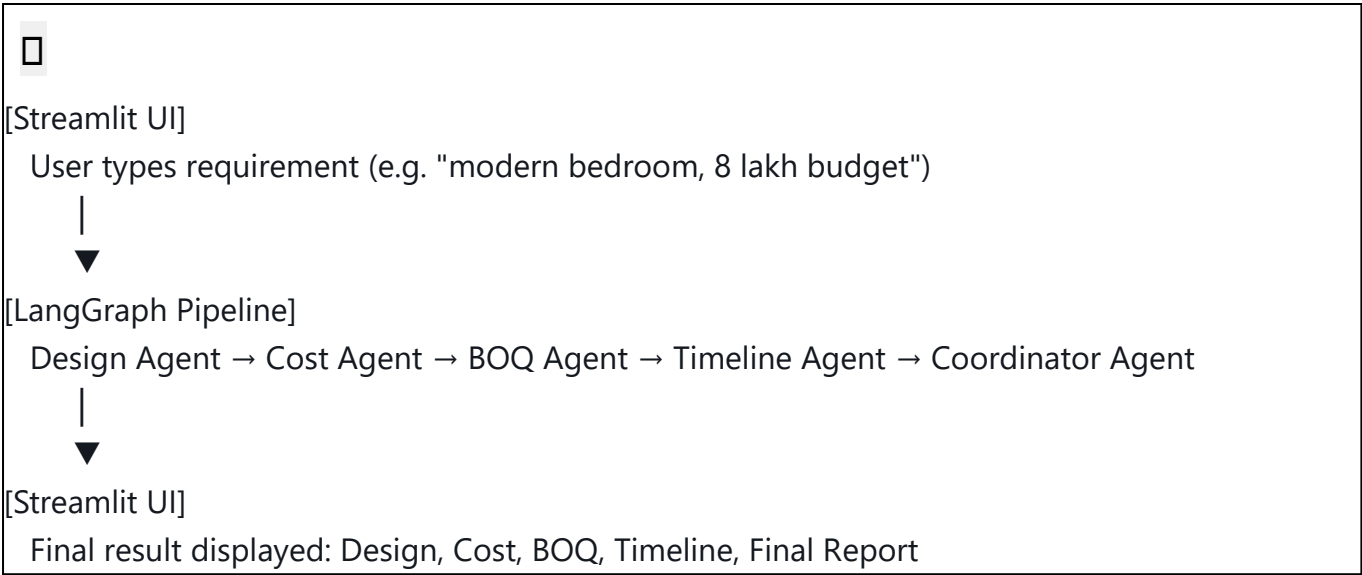
- User accounts / login / saved project history
- Payment or vendor integration

- Real-time 3D rendering of the design (may be considered as a future upgrade)
- Mobile app version
- Database persistence (results exist only for the current session)

4. Users

- **Primary user:** An interior designer or homeowner who wants a fast, rough project plan before committing to a full design engagement.
- **Secondary user (for this project's context):** Evaluators/instructors reviewing the FYP demo.

5. System Architecture / Flow



Each agent reads from and writes to a shared state object (`Interio0SSState`) as it moves through the LangGraph pipeline. The Coordinator Agent merges all outputs into a single final report.

6. Functional Requirements

6.1 Input

- A single text input field where the user describes their requirement in natural language (room type, style, budget, etc.).
- A "Generate Plan" button that triggers the pipeline.

6.2 Processing (Backend Agents)

Agent	Responsibility
Design Agent	Generates a design concept from the user's requirement (Claude API)
Cost Agent	Produces a rough cost estimate based on the design and budget
BOQ Agent	Generates a bill of quantities / material list
Timeline Agent	Produces an estimated project schedule
Coordinator Agent	Runs the LangGraph pipeline end-to-end and compiles the final report

6.3 Output (UI Display)

- **Design** → shown as a text block
- **Cost** → shown as a highlighted figure
- **BOQ** → shown as a table (`st.table()`)
- **Timeline** → shown as a simple list/steps
- **Final Report** → consolidated summary combining all sections
- Basic visual styling: headings, spacing, and color used to make sections easy to scan.

6.4 Error Handling (minimum viable)

- If the pipeline fails or an agent returns no result, the UI should show a clear message instead of a blank/broken screen.
- Empty input should not trigger the pipeline (basic validation on the text field).

7. Non-Functional Requirements

- **Simplicity:** No HTML/CSS/JS required to build the UI — Streamlit only.
- **Beginner accessibility:** Entire stack limited to skills the team can realistically learn within the project timeline (basic Python, 5–6 Streamlit commands, beginner LangGraph).
- **Response time:** Result should display within a reasonable wait after clicking "Generate" (exact time depends on Claude API latency across 4 chained agent calls).
- **No persistence required:** Session-based only; no database.

8. Tech Stack

Layer	Tool
Language	Python
Agent orchestration	LangGraph (StateGraph, nodes, edges)
AI model	Claude API (used by each agent)
UI	Streamlit
Version control	Git / GitHub (repo: InterioOS)

9. Team & Roles

Member	Name	Role	GitHub
1	Bareera Zayan	Design Agent	github.com/BareeraZayan
2	Atika Umar	Cost Estimation Agent	github.com/Atikacreator1
3	Muhammad Abdul Rehman Shakeel	BOQ Agent	github.com/abdulrehmanshakeel
4	Ammara Akhtar	Timeline Agent	github.com/AmmaraAkhtar
5	Amna Ali	Coordinator Agent + LangGraph Integration	—
6	Areeba Saleem	UI/UX (Streamlit)	github.com/AreebaSaleem95

Testing: No dedicated tester role at this scale. Member 5 and Member 6 jointly test their integration point (backend pipeline ↔ UI) during Week 4.

10. Project Timeline

Week	Milestone
1–2	Members 1–4 build their individual agents; Member 6 builds the basic Streamlit UI

Week	Milestone
	structure in parallel using dummy data
3	Member 5 integrates all agents into the LangGraph pipeline
4	Member 6 connects the real backend pipeline to the UI; final integration testing and polish

11. Deliverables

- Working Streamlit web app (`app.py`) with:
 - Requirement input + Generate button
 - Design, Cost, BOQ, Timeline, and Final Report sections
- LangGraph pipeline connecting all 5 backend agents (`coordinator_agent.py`, `design_agent.py`, etc.)
- Shared state schema (`InterioOSState`)
- GitHub repository (InterioOS) with each member's contribution on their respective branch

12. Success Criteria

- A user can type a requirement, click Generate, and see a complete Design + Cost + BOQ + Timeline + Final Report on screen without manual steps in between.
- The UI correctly reflects live output from Member 5's pipeline (not just dummy data) by the end of Week 4.
- No team member needs to write HTML/CSS/JS to complete their part of the UI.

13. Future Considerations (Not in Current Scope)

- Upgrading the UI to a 3D visualization of the design concept
- Saving project history per user
- Exporting the final report as a PDF